프로젝트 계획서

네이버 API 기반 위치 기반 식사 추천 서비스 (Web-view App)

작성일: 2026. 03. 06 | 학번/이름: 3102 / 김시우 | ver 1.0

---

1. 프로젝트 개요

프로젝트명	위치 기반 식사 추천 서비스 (Web-view App)
학번 / 이름	3102 / 김시우
담당 역할	Frontend (기능 구현 및 API 연동)
개발 기간	2026. 03. 06 ~ 2026. 04. 25 (총 8주)

2. 기획 배경 및 목적

해결하고자 하는 문제

점심·저녁 식사 시 메뉴 선택에 소요되는 의사결정 비용(결정 장애)은 일상적으로 반복되지만 기존 서비스는 검색어 입력을 전제로 하여 능동적 탐색을 요구한다. 위치 자동 감지 기반의 수동적(passive) 추천 흐름이 필요하다.

기획 의도 및 목표

네이버 지역 검색 API를 프록시 서버로 경유하여 신뢰도 높은 식당 데이터를 수집하고, 터치 한 번으로 메뉴를 결정할 수 있는 최소 기능 제품(MVP)을 구축한다. 이후 사용자 취향 데이터 축적을 기반으로 개인화 추천 엔진으로 고도화하는 것을 장기 목표로 설정한다.

3. 서비스 상세 내용

타겟 사용자

점심·저녁 메뉴 결정에 어려움을 겪는 직장인 및 대학(원)생, 낯선 지역에서 식당 탐색이 필요한 여행자

핵심 기능 (Core Features)

기능	상세 설명
위치 기반 식당 탐색	HTML5 Geolocation API로 사용자 위경도 수집 후 Naver Reverse Geocoding으로 행정동 단위 주소 변환, 상단 UI 표시
메뉴 추천 엔진	현재 운영 중인 식당 중 카테고리별 무작위 큐레이션. 거리순(500m/1km/2km), 평점순, 카테고리(한/중/일/양식) 필터 제공
상세 정보 딥링크	식당 선택 시 네이버 플레이스 앱 또는 모바일 웹으로 직접 연결, 현재 위치 기준 길 찾기 연동

차별점

기존 서비스 (네이버 지도, 카카오맵 등)

- 검색어 입력 또는 카테고리 탭 선택 필요
- 방대한 정보로 인한 선택 피로도 증가
- 능동적 탐색 전제 UX

본 서비스 (차별화 포인트)

- 입력 없이 위치 감지만으로 즉시 추천
- 단일 큐레이션 결과 제시로 결정 비용 최소화
- 수동적(passive) 추천 UX로 사용자 이탈 방지

4. 기술 스택 (Tech Stack)

Frontend ★ 본인 담당	Next.js (React) · TypeScript · Zustand (상태 관리) · Axios 위치 수집 로직, API 연동, 클라이언트 상태 관리 및 필터링 로직 전담
Backend	Node.js · Express 네이버 API 요청 대행 및 데이터 가공 프록시 서버 (팀원 담당)
App Shell ★ 협업	React Native WebView · Flutter WebView WebView 내 JavaScript Bridge 연동, 딥링크 intent/url scheme 분기 처리 협업
External API	Naver Search API (Local) · Naver Maps API · Naver Reverse Geocoding
DevOps / Infra	AWS EC2 · GitHub Actions · Docker · Redis (캐싱)
협업 툴	GitHub · Notion · Figma · Discord

5. 팀원 역할 및 책임 (R&R)

이름	담당 분야 (Role)	세부 업무 내용
조성현	Frontend (UI/UX)	Figma 화면 설계, 공통 컴포넌트 개발, 추천 화면 UI 구현, 반응형 웹 적용, 디자인 시스템 수립
김시우 ★	**Frontend (기능/연동)**	**HTML5 Geolocation 구현, Naver API 프론트 연동, 클라이언트 상태 관리 (Zustand), 필터링 로직, 딥링크 처리 및 WebView Bridge 연동**
이동현	Backend / PM	프로젝트 일정 관리, 네이버 API 프록시 서버 개발, Redis 캐싱 전략, AWS 배포 파이프라인 구축

6. 개발 일정 (WBS) — 김시우 담당 업무 기준

전체 기간: 2026. 03. 06 (금) ~ 2026. 04. 25 (금) / 8주

1~2주차	**기획 및 설계** (03/06 ~ 03/19) — 요구사항 정의 참여, 백엔드 API 명세서 검토, Geolocation 수집 흐름 설계, 클라이언트 상태 구조 초안 작성
3~4주차	**기초 개발** (03/20 ~ 04/02) — HTML5 Geolocation API 구현, 프록시 서버 연동 Axios 인스턴스 설정, Zustand 스토어 설계, 주소 변환 결과 프론트 파싱 로직 구현
5~6주차	**심화 개발** (04/03 ~ 04/16) — 거리·카테고리·평점 필터링 클라이언트 로직 구현, 네이버 플레이스 딥링크(intent/url scheme) 분기 처리, WebView JavaScript Bridge 연동, 무한 스크롤 또는 페이지네이션 구현
7주차	**테스트 및 디버깅** (04/17 ~ 04/21) — GPS 권한 거부 및 타임아웃 예외 처리, Android/iOS 딥링크 엣지 케이스 수정, API 응답 오류 처리 로직 보완, QA 진행
8주차	**마무리 및 발표** (04/22 ~ 04/25) — 배포 환경 프론트 성능 최적화(Lighthouse 점검), 최종 통합 테스트, 시연 시나리오 작성, 보고서 기능 구현 파트 작성

7. 최종 산출물 목록 (Deliverables)

구분	항목
문서 산출물	요구사항 명세서, Figma 화면 설계서, DB 설계도(ERD), API 명세서

개발 산출물	소스코드 (GitHub Repository), 배포된 서비스 접속 URL, AOS/iOS WebView 빌드 파일
발표 산출물	최종 결과 보고서, 시연 영상 (선택)

8. 기대 효과 및 발전 방향

기대 효과

- **사용자 편의성:** 검색어 입력 없이 터치 한 번으로 식사 메뉴 결정 프로세스를 단축하여 결정 피로도 해소
- **개발 효율성:** 네이버의 검증된 DB를 활용하여 식당 정보 수집·갱신 비용 절감, 데이터 신뢰성 확보
- **보안 안정성:** 프록시 서버 구조로 API 키 클라이언트 노출 차단, 인메모리 캐싱으로 API 쿼터 절약

추후 발전 방향

- 사용자의 선택 이력 및 평가 데이터를 축적하여 개인화 AI 추천 모델로 고도화
- 동행인 수 및 예산 입력 기반의 그룹 추천 기능 추가
- 앱스토어(Google Play / App Store) 정식 배포 및 푸시 알림 연동

9. 기술적 고려사항 및 해결 전략

구분	리스크	해결 전략
GPS 권한 UX	진입 시 즉시 권한 요청으로 사용자 이탈 및 브라우저 정책 제한	권한 요청 시점을 사용자 액션(버튼 클릭) 이후로 설정, GPS 거부·타임아웃 시 수동 주소 입력 fallback UI 제공
WebView 딥링크	외부 앱 호출 시 intent/url scheme 처리 미흡으로 앱 미설치 상황 대응 실패	Android는 intent://, iOS는 universal link 방식으로 분기 처리, fallback URL로 모바일 웹 연결 설정
API 응답 오류	네트워크 불안정 또는 API 장애 시 빈 화면 노출	Axios 인터셉터 기반 오류 처리, 재시도(retry) 로직 및 스켈레톤 UI 적용으로 사용자 경험 유지
상태 관리	필터·위치·추천 데이터 간 상태 불일치	Zustand 중앙 스토어로 전역 상태 단일화, 액션 기반 상태 업데이트 패턴 강제